EXPERIMENT NO. 4

FREQUENCY DOMAIN IMAGE FILTERING USING FOURIER TRANSFORM

In [3]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. LOAD GRAYSCALE IMAGE
image_path = input("Enter the path of grayscale image: ")
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image could not be loaded!")
    exit()

print("Image loaded successfully!")
print("Image Shape:", image.shape)


# 2. DISPLAY ORIGINAL IMAGE
plt.imshow(image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()


# 3. COMPUTE DISCRETE FOURIER TRANSFORM (DFT)
dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
print("DFT calculated successfully.")


# 4. SHIFT ZERO FREQUENCY TO CENTER
dft_shift = np.fft.fftshift(dft)
print("Frequency spectrum shifted to center.")


# 5. CALCULATE MAGNITUDE SPECTRUM
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1)


# 6. DISPLAY FREQUENCY SPECTRUM
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()


# 7. CREATE LOW-PASS FILTER
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2
radius = 50

low_pass_mask = np.zeros((rows, cols, 2), np.uint8)
cv2.circle(low_pass_mask, (ccol, crow), radius, (1, 1), -1)

print("Low-Pass Filter created.")
```

```
# 8. APPLY LOW-PASS FILTER
low_pass_dft = dft_shift * low_pass_mask


# 9. INVERSE FOURIER TRANSFORM FOR LOW-PASS
low_shift = np.fft.ifftshift(low_pass_dft)
low_image = cv2.idft(low_shift)

low_image = cv2.magnitude(
    low_image[:, :, 0],
    low_image[:, :, 1]
)

low_image = cv2.normalize(
    low_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)


# 10. DISPLAY LOW-PASS FILTERED IMAGE
plt.imshow(low_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")
plt.show()


# 11. CREATE HIGH-PASS FILTER
high_pass_mask = np.ones((rows, cols, 2), np.uint8)
cv2.circle(high_pass_mask, (ccol, crow), radius, (0, 0), -1)

print("High-Pass Filter created.")


# 12. APPLY HIGH-PASS FILTER
high_pass_dft = dft_shift * high_pass_mask


# 13. INVERSE FOURIER TRANSFORM FOR HIGH-PASS
high_shift = np.fft.ifftshift(high_pass_dft)
high_image = cv2.idft(high_shift)

high_image = cv2.magnitude(
    high_image[:, :, 0],
    high_image[:, :, 1]
)

high_image = cv2.normalize(
    high_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)


# 14. DISPLAY HIGH-PASS FILTERED IMAGE
plt.imshow(high_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")
plt.show()


# 15. DISPLAY ALL RESULTS TOGETHER
```

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(low_image, cmap="gray")
plt.title("Low-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(high_image, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(low_pass_mask[:, :, 0], cmap="gray")
plt.title("Low-Pass Mask")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(high_pass_mask[:, :, 0], cmap="gray")
plt.title("High-Pass Mask")
plt.axis("off")

plt.tight_layout()
plt.show()


# 16. SAVE OUTPUT IMAGES
cv2.imwrite("original_image.jpg", image)

cv2.imwrite(
    "magnitude_spectrum.jpg",
    cv2.normalize(
        magnitude_spectrum, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)
)

cv2.imwrite("low_pass_filtered.jpg", low_image)
cv2.imwrite("high_pass_filtered.jpg", high_image)
```

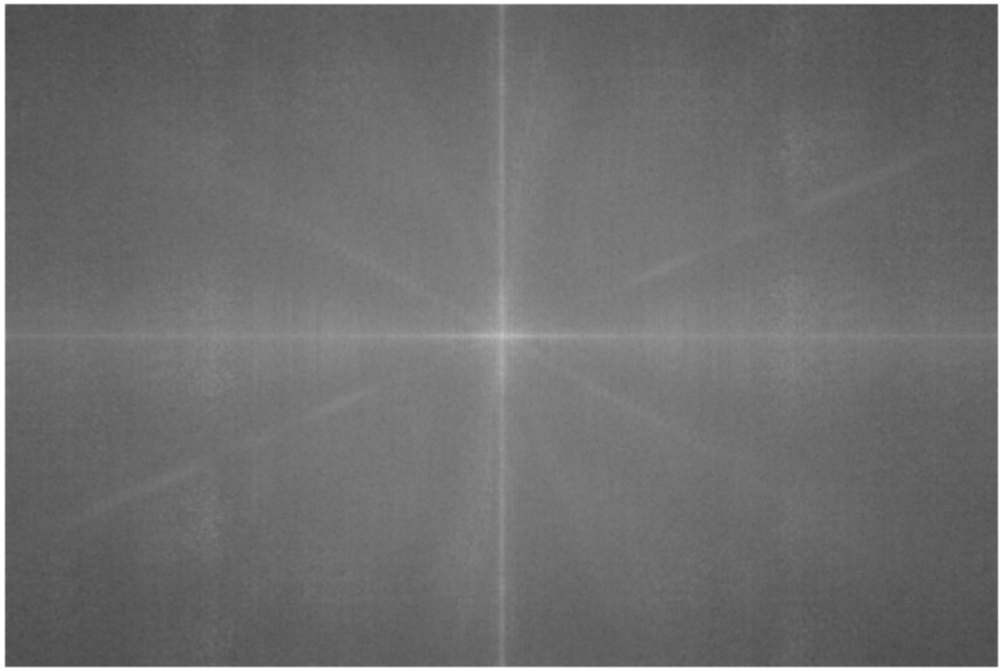
```
Image loaded successfully!
Image Shape: (1024, 1536)
```

Original Grayscale Image



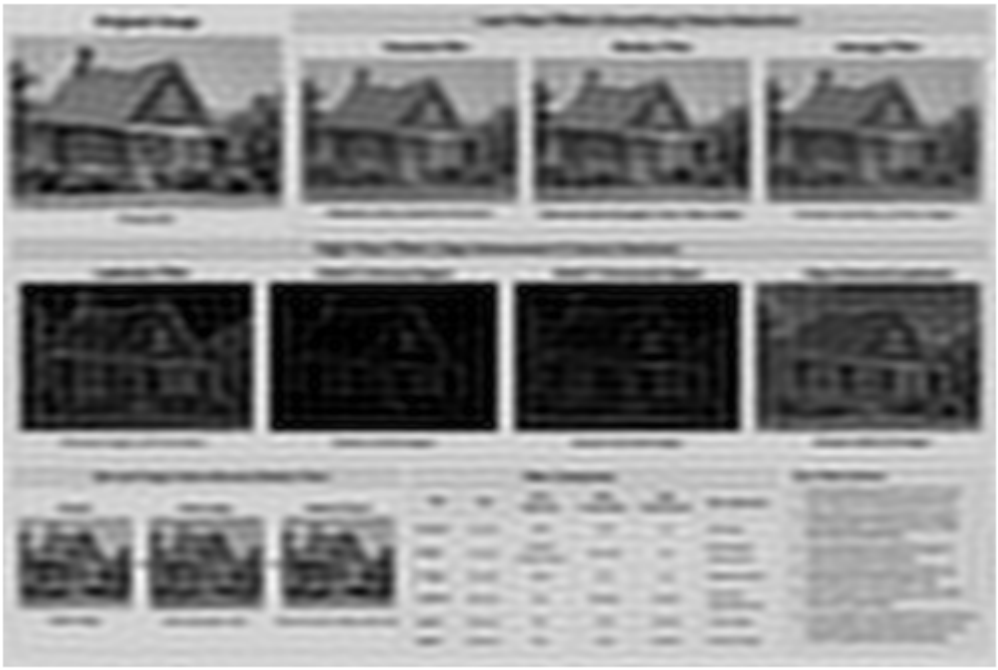

```
DFT calculated successfully.
Frequency spectrum shifted to center.
```

Magnitude Spectrum



```
Low-Pass Filter created.
```

Low-Pass Filtered Image



```
High-Pass Filter created.
```

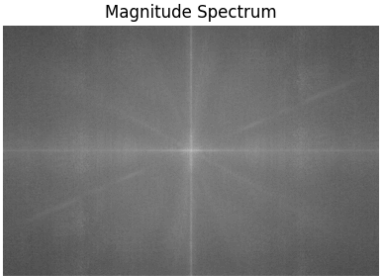
High-Pass Filtered Image



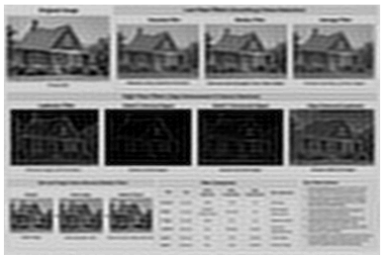
Original Image



Magnitude Spectrum



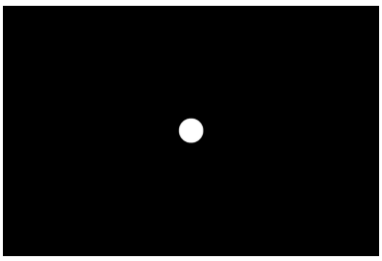
Low-Pass Filter



High-Pass Filter



Low-Pass Mask



High-Pass Mask



Out[3]: True